YUVA INTERN

EMBEDDED SYSTEMS INTERNSHIP | FINAL TECHNICAL REPORT (WEEK 4)

IoT-Based Smart Environmental Monitoring and Alert System

From requirements to verified firmware: system integration, acceptance testing and project close-out

Week 4 Task

System Integration and Final Testing

Item	Details
Student Name	Pilla Yashwanth (Yuva Intern)
Email	Yaswanthvasu79@gmail.com
Instuite	Ignite Embedded systems
Internship Organization	Yuva intern
Report Date	27/10/2026
Final Release	Firmware v3.0.0 (frozen), single file smart_env_monitor.c
Test Platform	Simulated hardware on a PC. No physical hardware was used in this project.

Table of Contents

Executive Summary 3
1. Introduction and Scope of Week 4 3
2. Final System Overview 3
2.1 Software Structure 4
3. Integration Approach 5
3.1 Release Configuration 5
4. System-Level Acceptance Test 6
4.1 Is the Acceptance Test Strong Enough? 6
5. Requirements Compliance 7
6. Final Verification Results 8
6.1 Performance Result (Week 3, carried forward) 9
7. Defects and Lessons Learned 9
8. Hardware Validation Plan (Not Yet Performed) 9
8.1 Porting Checklist 10
9. Limitations and Risks 10
10. Future Work 11
11. Final Submission Checklist 11
11.1 Before You Submit: Fill In 11
12. Conclusion 12
Appendix A: Acceptance Test Output 13
Appendix B: Final Self-Test Summary 13
Appendix C: 30-Day Soak Report 14
Appendix D: Glossary 14

> If the table of contents appears empty, right-click it in Word and choose Update Field, then Update entire table.

Executive Summary

This project designed, implemented and tested the firmware of a low-cost environmental monitoring node. The node measures temperature, humidity and a relative air-quality indication, shows them on an OLED display, raises a local LED and buzzer alert when configurable limits are exceeded, and reports to a remote dashboard over Wi-Fi. It keeps monitoring and alerting if the network fails.

Table 1: Project at a glance

Week	Outcome
1 Plan	26 functional and 13 non-functional requirements, hardware block diagram, software flowchart, fault analysis, traceability matrix.
2 Build	Complete firmware in one C file with timer and UART interrupts and a simulated hardware layer; 20 self-tests.
3 Debug and optimise	5 defects found and fixed with regression tests; test quality measured by coverage (99.3%) and mutation testing (16 of 16); firmware work reduced by 48%, display traffic by 78.5%, behaviour unchanged.
4 Integrate and verify	System-level acceptance run: 14 scenario steps, all 26 functional requirements and 5 non-functional requirements demonstrated, 0 failures; 30-simulated-day soak with 0 violations; final verification, hardware validation plan and close-out.

1. Introduction and Scope of Week 4

Week 4 closes the project. Because the official Week 4 task text was not available, it is interpreted from the roadmap as **System Integration and Final Testing**. With no physical board, integration means bringing the finished firmware modules, the interrupt routines and the (simulated) hardware layer together and testing the **complete system as one**, against the requirements written in Week 1.

- Integrate all modules and interfaces into one frozen release (firmware v3.0.0).
- Run a continuous end-to-end acceptance scenario and check every Week 1 requirement.
- Run the final verification campaign and an extended soak test.
- Document what is verified, what is not, and exactly how to finish the verification on real hardware.
- Provide the complete submission package.

2. Final System Overview

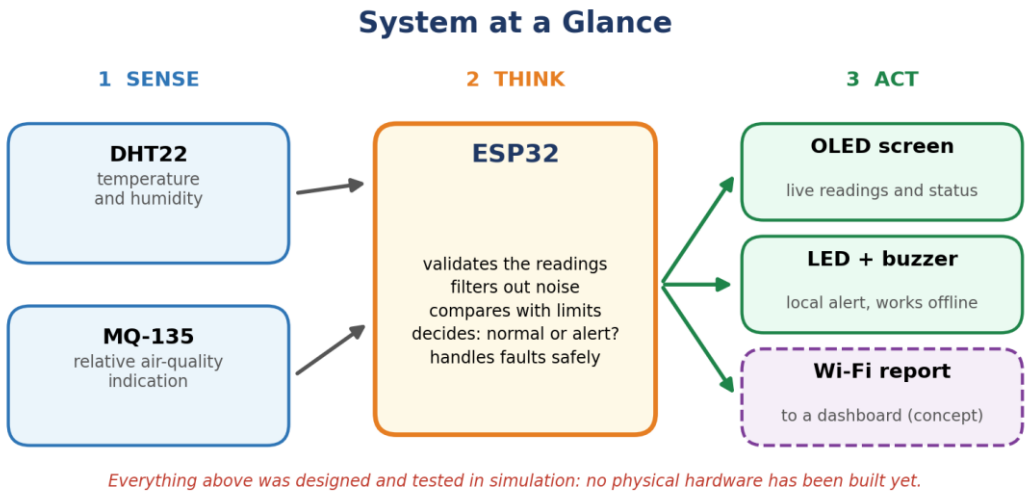


Figure 1: The system in three steps: sense, think, act

Sense: a DHT22 provides temperature and humidity every 2 s; an MQ-135 gas sensor, read through a voltage divider on a Wi-Fi-safe ADC channel, provides a relative air-quality index every second. **Think:** the ESP32 firmware

validates each sample, filters noise, compares values with configurable limits using confirmation and hysteresis, and handles faults. **Act:** the OLED shows readings and status, the LED and buzzer signal alerts locally, and a JSON message is sent every 15 s over Wi-Fi.

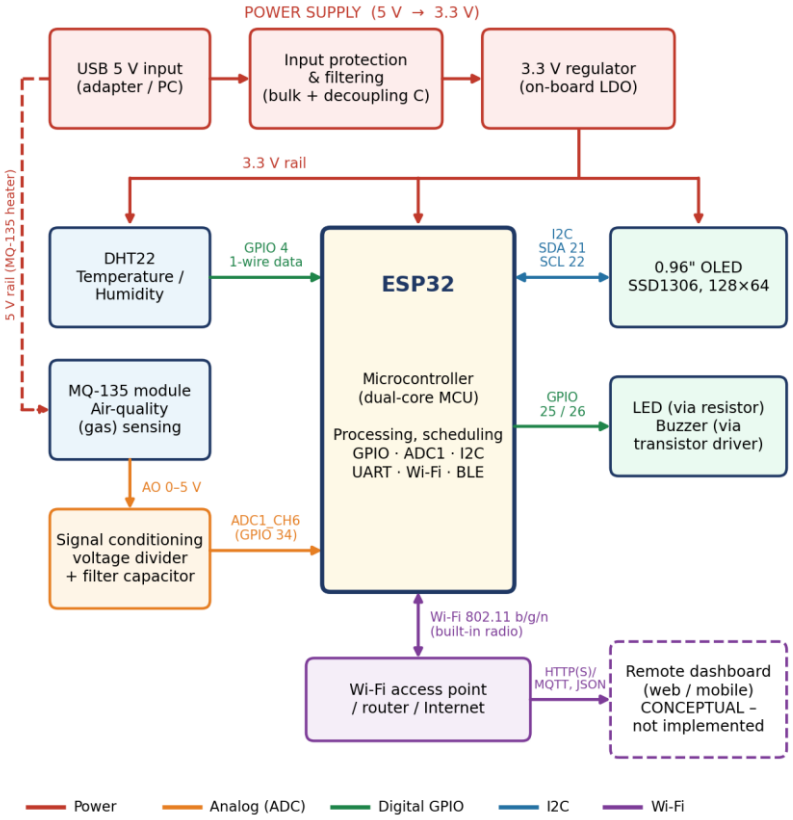


Figure 2: Hardware block diagram (Week 1)

2.1 Software Structure

The firmware keeps interrupt work and main-loop work apart. The timer interrupt only advances a millisecond tick (Week 3 optimisation); a cooperative main loop runs each task from its deadline. A UART interrupt feeds a ring buffer for a command interface. All hardware access sits behind a hardware abstraction layer, so porting means replacing one section of the file.

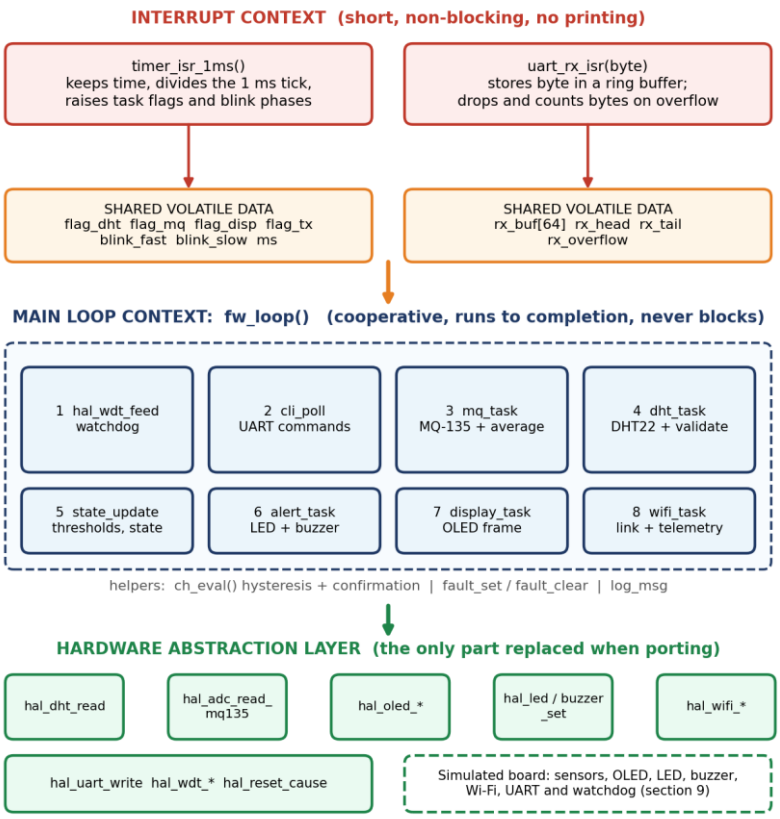


Figure 3: Firmware architecture (Week 2; the ISR no longer raises task flags after Week 3)

3. Integration Approach

Table 2: Integration steps and evidence

No.	Integration step	Evidence	Week
1	Hardware boundary defined: 15 HAL functions are the only way the firmware touches hardware	Section 3 of the source; simulated board in section 9	2
2	Interrupts integrated with the main loop (tick and UART ring buffer)	T27 (ISR changes only the tick), T16, T29 (fuzzing)	2, 3
3	Sensor, threshold, state, alert, display and Wi-Fi modules integrated in a fixed loop order	32 unit and integration tests	2, 3
4	Fault paths integrated (sensor, display, Wi-Fi, transmission, reset, watchdog)	T09 to T19, T24, T32	2, 3
5	Whole system exercised in one continuous scenario with state carried between steps	Acceptance run, 14 steps (Section 4)	4
6	Long-duration behaviour with random faults	Soak runs of 24 h, 7 days and 30 days	3, 4
7	Release frozen and verified at four optimisation levels	Section 6	4

3.1 Release Configuration

Table 3: Final release

Item	Value
Firmware	smart_env_monitor.c, version 3.0.0-sim, about 1,800 lines, C99, no external libraries
Build	gcc -std=c99 -Wall -Wextra -Wpedantic -O2 -o smart_env_monitor smart_env_monitor.c -lm

Item	Value
Run modes	(none) self-tests, verbose, demo, bench, soak
Acceptance test	acceptance_test.c (includes the firmware; runs the end-to-end scenario)
Dependencies	A C compiler. Valgrind and cppcheck were used for analysis only.

4. System-Level Acceptance Test

Unit tests start every test from a clean state. The acceptance test is different: **one continuous run** of about 8.5 simulated minutes in which events happen one after another and the system must recover from each without intervention. Every check is tagged with the requirement it demonstrates.

Table 4: Acceptance scenario

Step	Event	What must happen
1	Power on	Outputs off, initialisation complete, boot logged, warm-up starts (FR-01, 24, 26)
2	Warm-up then normal operation	Baseline captured, readings acquired, OLED shows values with units, Wi-Fi up (FR-02 to 05, 07 to 10, 16)
3	Periodic telemetry	Messages every 15 s with device id, sequence, data and fault flags (FR-17, 18)
4	Sensor glitches	One over-limit sample ignored; one implausible spike rejected (FR-06, 12)
5	Temperature alert, hysteresis, clear	Alert within 4 s, LED and buzzer on, stays on inside band, clears below limit (FR-11 to 15)
6	Humidity alert	Raised and cleared (FR-11, 14)
7	DHT22 unplugged during an alert	Fault declared, alert based on invalid data cleared, other sensor keeps running, automatic recovery (FR-06, 21)
8	Wi-Fi lost during an alert	Alert works offline, display shows status, automatic reconnection (FR-19, 20)
9	OLED removed	Alert and telemetry continue; display recovers (FR-22)
10	MQ-135 stuck, then poor air	Fault shown; recovery; air alert raised and cleared (FR-04, 21)
11	UART commands	STATUS, runtime threshold change, mute (FR-13, 23, 24)
12	Transmission failure	Failure flagged, link stays up, recovery (FR-17)
13	Power interruption during an alert	Outputs off immediately, warm-up restarts, reset cause logged, alert re-evaluated (FR-01, 11, 26)
14	Firmware lock-up	Watchdog reset, cause logged, system running again (FR-25)

Result: 31 requirement identifiers exercised (all 26 functional and 5 non-functional), 0 failed. The full compliance list is in Appendix A.

4.1 Is the Acceptance Test Strong Enough?

As in Week 3, the test was checked by injecting defects into a copy of the firmware. The first versions of the scenario missed some of them, and the scenario was extended twice. It now detects all seven injected defects.

Table 5: Injected defects and acceptance detection

Injected defect	Detected by	Note
Hysteresis removed	FR-12	Detected at once
Alert not cleared when its sensor fails	FR-06, FR-21	First missed: no alert was active when the sensor was unplugged. Step 7 now starts with an active alert.
Outputs not forced off at boot	FR-26	Detected at once
Wi-Fi never retries	FR-17, 20, 22	Detected at once
Watchdog never fed	16 identifiers	Detected at once

Injected defect	Detected by	Note
Confirmation count ignored	FR-12	First missed: the alert flashed and cleared unseen. Step 4 now checks the outputs never switch on.
Spike filter removed	FR-06	First missed. Step 4 now injects a spike.

5. Requirements Compliance

Table 6: Functional requirements: evidence and status

ID	Requirement	Evidence (acceptance step; unit tests)	Status
FR-01	Initialise hardware at start-up	Steps 1, 13; T01, T18	Verified (simulation)
FR-02, 03	Acquire temperature and humidity	Step 2; T02	Verified (simulation)
FR-04	Acquire air-quality reading	Steps 2, 10; T02, T07	Verified (simulation)
FR-05	Process sensor data (average, convert)	Step 2; T02, T07	Verified (simulation)
FR-06	Validate readings	Steps 4, 7; T09, T10, T19	Verified (simulation)
FR-07	MQ-135 warm-up	Steps 1, 2; T01, T08	Verified (simulation)
FR-08	Show readings on OLED	Step 2; T02	Verified (simulation)
FR-09	Show air-quality status	Steps 2, 10; T02, T07	Verified (simulation)
FR-10	Show alert and Wi-Fi status	Steps 2, 5, 8; T02, T13	Verified (simulation)
FR-11	Compare with configurable thresholds	Steps 5, 6, 13; T03, T06	Verified (simulation)
FR-12	Hysteresis and confirmation	Steps 4, 5; T04 to T06	Verified (simulation)
FR-13, 14	Activate and clear LED and buzzer	Steps 5, 6, 11; T03, T04	Verified (simulation)
FR-15	Distinct alert and fault patterns	Steps 5, 7; T03, T09	Verified (simulation)
FR-16	Establish Wi-Fi	Step 2; T01, T12	Verified (simulation)
FR-17, 18	Periodic telemetry with required fields	Steps 3, 12; T14, T15	Verified (simulation)
FR-19	Local monitoring when Wi-Fi is down	Step 8; T12, T13	Verified (simulation)
FR-20	Automatic reconnection (no blocking)	Step 8; T12, T13	Verified (simulation)
FR-21	Detect and indicate sensor faults	Steps 7, 10; T09, T10	Verified (simulation)
FR-22	OLED failure does not stop the system	Step 9; T11, T24	Verified (simulation)
FR-23	Configurable thresholds	Step 11; T16, T25	Verified (simulation)
FR-24	Serial diagnostics	Steps 1, 11; T16	Verified (simulation)
FR-25	Watchdog recovery	Step 14; T17	Verified (simulation)

ID	Requirement	Evidence (acceptance step; unit tests)	Status
FR-26	Outputs off at reset	Steps 1, 13; T01, T17, T18	Verified (simulation)

Table 7: Non-functional requirements: status

ID	Requirement	Evidence or reason	Status
NFR-01	Reliability	Automatic recovery from every fault in the acceptance run; 30-day soak with 0 violations and 0 watchdog resets	Verified (simulation)
NFR-02	Fault isolation	One failed subsystem never stopped the others (steps 7 to 10)	Verified (simulation)
NFR-03	Maintainability	Single-responsibility modules, central configuration, 99.3% line coverage	By inspection
NFR-04	Scalability	Device id in every message; multiple nodes not tested	Partly verified
NFR-05	Usability	OLED text uses units and plain words; legibility at distance needs the real display	Partly verified
NFR-06	Performance	Instruction counts in simulation; timing on the ESP32 not measured	Partly verified
NFR-07	Responsiveness	Alert in 3000 ms after the step (limit 4000 ms); network never delays it	Verified (simulation)
NFR-08	Power efficiency	Needs current measurement	Not verified
NFR-09	Modularity	HAL boundary; modules replaceable	By inspection
NFR-10	Portability	HAL isolates hardware; port not performed	By inspection
NFR-11	Safety	Outputs off by default (tested); low-voltage design; hardware review pending	Partly verified
NFR-12	Security	Real network and encryption not implemented	Not verified
NFR-13	Environmental range	Needs real parts and conditions	Not verified

6. Final Verification Results

Table 8: Verification of the final release

Check	Result
Self-test suite (32 tests)	163 checks, 0 failed, at -O0, -O2, -O3 and -Os
Compiler warnings	0 with -Wall -Wextra -Wpedantic -Wshadow -Wconversion at four optimisation levels
System acceptance run	14 steps, 31 requirement identifiers, 0 failed
Line coverage of firmware	99.3% (404 of 407 lines)
Mutation testing, unit suite	16 of 16 injected defects caught
Mutation testing, acceptance run	7 of 7 injected defects detected
Memory and undefined behaviour	AddressSanitizer, UBSan: no reports. Valgrind: 0 errors (Week 2 baseline: 12606)
Static analysis (cppcheck)	0 findings
Fuzzing	100000 random bytes into the UART interrupt in the suite (400000 in the Week 3 hunt): no failure
Soak, 30 simulated days (720 h)	12,847 random events, 223 injected resets, 44,879 telemetry messages, 0 invariant violations, 0 watchdog resets

6.1 Performance Result (Week 3, carried forward)

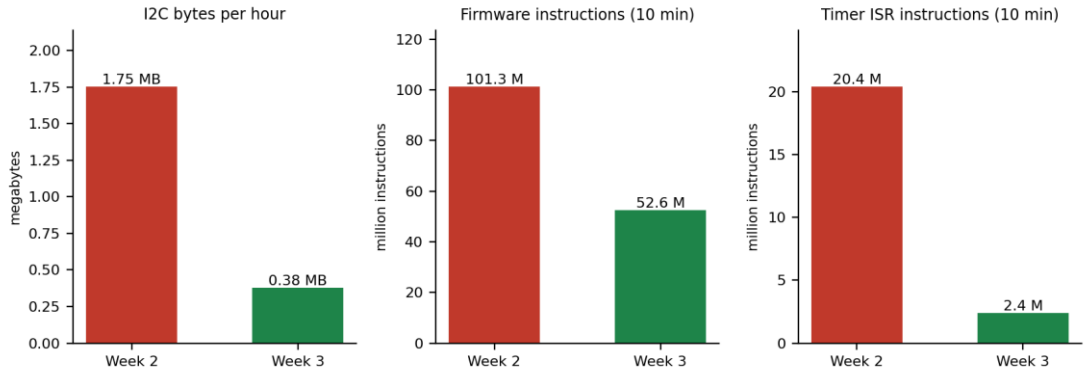


Figure 4: Week 2 versus Week 3: firmware instructions, timer ISR instructions and display bytes (simulation)

Firmware instructions fell by 48% (101.3 M to 52.6 M for a 10-minute scenario), timer-interrupt work by 88%, and display bus traffic by 78.5% (1.75 MB to 0.38 MB per hour), while a fingerprint of every alert, LED and state change stayed identical. These are x86 host measurements and show relative change, not ESP32 timing.

7. Defects and Lessons Learned

Table 9: Defects found during the project

ID	Defect	Found by	Week
D1	Display change detection compared uninitialised memory (screen rewritten every second at -O0)	Valgrind	3
D2	32-bit tick rollover after 49.7 days: state returned to warm-up, deadlines unsafe	Time-warp test	3
D3	Changing a threshold silenced an active alert	Targeted probing	3
D4	Static-analysis and conversion findings	cppcheck, -Wconversion	3
D5	Fault cleared late in a loop pass took effect one pass later	7-day soak	3

- **Passing tests are not proof.** The Week 2 suite passed while D1 to D3 were present. Different tools find different defects: Valgrind found D1, which the sanitizers missed.
- **Test the tests.** Injecting defects showed that eight of my new tests and scenarios were too weak at first. Each was strengthened until the injected defect was caught.
- **Long-running defects need time-travel.** D2 appears only after 49.7 days. A simulator that can jump the clock finds it in milliseconds.
- **Measure before optimising.** The profile showed that a function running every millisecond (state evaluation) was 25% of the work and almost always idle.
- **Simulate honestly.** A simulation proves the logic. It does not prove timing, electronics or the environment, and the report must say so.

8. Hardware Validation Plan (Not Yet Performed)

This plan turns the open items into a checklist for the day a board is available. Every row is **planned, not done**. It is ordered so that each step makes the next one safe.

Table 10: Hardware bring-up and validation checklist

No.	Activity	Pass criterion	Closes
H-01	Check wiring against the Week 1 pin map; fit the MQ-135 divider before connecting AO to GPIO34	Wiring matches; AO never reaches the pin directly	NFR-11
H-02	Measure supply current at 5 V (idle, Wi-Fi transmit peak)	Within the 1 A supply; compare with the 0.45 to 0.55 A estimate	NFR-08
H-03	Measure the 3.3 V rail under load, look for dips during Wi-Fi bursts	Stable; no brown-out resets	NFR-08
H-04	I2C scan	OLED answers at 0x3C or 0x3D	FR-08
H-05	Display a test frame; run firmware display task	Six lines readable; per-line updates work	NFR-05
H-06	Compare DHT22 with a reference thermometer and hygrometer	Record error; reading every 2 s without failures	FR-02, 03
H-07	Check divider output with a multimeter; log ADC counts in clean air	Voltage at most 3.3 V; counts stable	FR-04
H-08	Log MQ-135 for several minutes after power-on; choose the warm-up time and baseline	Warm-up value replaces the 10 s simulation value	FR-07
H-09	Toggle a spare pin in the timer ISR; measure on a scope or logic analyser	Period 1 ms; ISR duration recorded	NFR-06
H-10	Toggle a pin per DHT22 sample; measure jitter	Sample period 2 s with small jitter	NFR-06
H-11	Warm the sensor safely above the limit, then cool	Alert within 4 s; hysteresis as designed; record latency	FR-11 to 15, NFR-07
H-12	Join a real network; switch the router off and on; receive messages on a test server	Offline mode, reconnection, messages every 15 s	FR-16 to 20
H-13	Unplug DHT22 and OLED in turn	Fault indication and automatic recovery	FR-21, 22
H-14	Power-cycle during an alert; provoke a lock-up if possible	Outputs off at boot; watchdog recovery	FR-25, 26
H-15	24-hour unattended run	Record resets, reconnections, drift	NFR-01

8.1 Porting Checklist

- Replace section 9 (simulated hardware) with real drivers: DHT library on GPIO4, analogRead on GPIO34 (ADC1), SSD1306 on I2C GPIO21/22, GPIO25/26 for LED and buzzer driver, WiFi client with timeouts, Serial, task watchdog, reset-reason function.
- Drive timer_isr_1ms from a 1 ms hardware timer. Keep it as short as it is.
- Review the two remaining interrupt-shared items (tick and UART ring buffer) for atomicity on the target.
- Replace placeholder values (warm-up time, air-quality percentages, thresholds, spike limits) with values from H-06 to H-08.

9. Limitations and Risks

- **Simulation only.** Timing, current, sensor behaviour, display and network behaviour on real parts are unknown. Instruction counts are x86 host counts and the I2C time is calculated.
- **MQ-135.** Non-selective, needs warm-up and calibration; it is a relative indicator. The system must not be used as a safety or life-protection detector.
- **Placeholder values.** Thresholds, air-quality percentages, spike limits and the 10 s warm-up are initial proposals.
- **Interrupt races** cannot be seen in a single-threaded simulation.
- **Dashboard and security** are conceptual: no cloud service, TLS or authentication was implemented.

- **Single node.** Multi-node operation is supported by a device id but not tested.

10. Future Work

The following are ideas for later work. None is implemented.

Improvement	Benefit
Cloud database and dashboard	Long-term records, graphs, alerts to phones
Mobile application	Notifications and viewing on the move
Better air-quality sensor (digital, selective)	Meaningful gas or CO2 values
Non-volatile configuration	Thresholds survive a restart
Local data buffering	No data loss during Wi-Fi outages
Battery or solar operation, sleep modes	Portable use (needs a sensor without a continuous heater)
Secure over-the-air updates	Safe remote maintenance
TLS and authentication	Protected data path

11. Final Submission Checklist

Table 11: Deliverables across the project

Week	File	Content	Done
1	Week1_Technical_Report_IoT_Environmental_Monitor.docx	Requirements and architecture report	Yes
1	Figure1_Block_Diagram.png, Figure2_Software_Flowchart.png	Diagrams	Yes
2	smart_env_monitor.c (v2), README.md	Firmware and instructions	Yes
2	Week2_Technical_Report_Firmware_Development.docx	Firmware report	Yes
2	test_results.txt, demo_output.txt	Captured output	Yes
3	smart_env_monitor.c (v3), README.md	Debugged and optimised firmware	Yes
3	Week3_Technical_Report_Debugging_Optimization.docx	Debugging and optimisation report	Yes
3	test_results_week3.txt, week3_analysis_tools.zip	Evidence and tools	Yes
4	Week4_Final_Technical_Report.docx	This report	Yes
4	smart_env_monitor.c (final), acceptance_test.c	Final release and acceptance test	Yes
4	acceptance_log.txt, final_test_results.txt, soak_30d.txt	Final evidence	Yes
All	Main README.md, docs/images/	Repository front page	Yes

11.1 Before You Submit: Fill In

- Registration number, college, organisation, supervisor name and date on each report cover.
- Your repository address in the main README quick-start and the license line.
- A screenshot of your own terminal showing the final test result (Tests passed: 32/32) and the acceptance result (Requirements exercised: 31 failed: 0).
- Update the contents page in Word (right-click, Update Field).

12. Conclusion

Over four weeks the project moved from a written specification to firmware that was implemented, debugged, optimised and tested against every functional requirement. The final system passes 32 unit and integration tests, a continuous 14-step acceptance scenario covering all 26 functional requirements, and a 30-day randomised fault-injection soak, with no memory errors, no compiler warnings and 99.3% coverage. The test methods were themselves tested by injecting defects, which exposed and corrected weak tests along the way.

The limits are as important as the results. Everything was verified in simulation. The real electronics, timing, power draw and environment remain to be measured, and Section 8 gives the checklist for doing so. The firmware is structured so that this step changes only the hardware layer.

Appendix A: Acceptance Test Output

```
STEP 01  Power on: safe outputs, initialisation, warm-up starts         [t=0 s]
STEP 02  Warm-up, then normal operation, display and Wi-Fi              [t=5 s]
STEP 03  Periodic telemetry                                            [t=13 s]
STEP 04  Sensor glitches: one over-limit sample and one implausible spike [t=48 s]
STEP 05  Temperature alert (36 C), hysteresis, clear                   [t=78 s]
STEP 06  Humidity alert and clear                                      [t=113 s]
STEP 07  DHT22 unplugged during an alert, then reconnected             [t=131 s]
STEP 08  Wi-Fi lost during an alert, then restored                     [t=184 s]
STEP 09  OLED removed and restored                                     [t=231 s]
STEP 10  MQ-135 stuck, recovery, poor air alert                        [t=319 s]
STEP 11  UART interface: STATUS, SET, MUTE                             [t=380 s]
STEP 12  Transmission failure and recovery                             [t=397 s]
STEP 13  Power interruption during an alert                            [t=452 s]
STEP 14  Firmware lock-up recovered by the watchdog                    [t=484 s]
REQUIREMENT COMPLIANCE (system-level acceptance run)
FR-01    PASS  (2/2 checks)
FR-02    PASS  (1/1 checks)
FR-03    PASS  (1/1 checks)
FR-04    PASS  (2/2 checks)
FR-05    PASS  (1/1 checks)
FR-06    PASS  (2/2 checks)
FR-07    PASS  (2/2 checks)
FR-08    PASS  (1/1 checks)
FR-09    PASS  (1/1 checks)
FR-10    PASS  (3/3 checks)
FR-11    PASS  (4/4 checks)
FR-12    PASS  (2/2 checks)
FR-13    PASS  (2/2 checks)
FR-14    PASS  (3/3 checks)
FR-15    PASS  (1/1 checks)
FR-16    PASS  (1/1 checks)
FR-17    PASS  (2/2 checks)
FR-18    PASS  (1/1 checks)
FR-19    PASS  (1/1 checks)
FR-20    PASS  (1/1 checks)
FR-21    PASS  (2/2 checks)
FR-22    PASS  (1/1 checks)
FR-23    PASS  (1/1 checks)
FR-24    PASS  (2/2 checks)
FR-25    PASS  (1/1 checks)
FR-26    PASS  (2/2 checks)
NFR-01   PASS  (5/5 checks)
NFR-02   PASS  (1/1 checks)
NFR-04   PASS  (1/1 checks)
NFR-05   PASS  (1/1 checks)
NFR-07   PASS  (1/1 checks)
Requirements exercised: 31   failed: 0   simulated time: 514 s
```

Appendix B: Final Self-Test Summary

```
T01  Start-up sequence, safe outputs, warm-up       PASS
T02  Normal operation and OLED content              PASS
T03  Temperature alert and response time            PASS
T04  Hysteresis (no chatter near threshold)         PASS
T05  Isolated over-limit sample ignored            PASS
T06  Humidity and low-temperature alerts            PASS
T07  Air-quality alert, recovery                    PASS
T08  Warm-up suppresses air-quality alert           PASS
T09  DHT22 disconnected, recovery                   PASS
T10  MQ-135 disconnected, recovery                  PASS
T11  OLED absent at start, later recovery           PASS
T12  Wi-Fi unavailable, back-off, reconnect         PASS
T13  Wi-Fi drops during operation                   PASS
T14  Telemetry interval and payload                 PASS
T15  Transmission failure handling                  PASS
T16  UART commands and ring buffer                  PASS
T17  Watchdog feeding and lock-up reset             PASS
T18  Power interruption (brown-out) restart         PASS
T19  Spike and out-of-range data handling           PASS
T20  Display bus traffic (per-line writes)          PASS
T21  Clock rollover: state and air alert            PASS
T22  Clock rollover: periodic task timing           PASS
T23  Threshold change keeps an active alert         PASS
T24  OLED removed during operation                  PASS
T25  UART: remaining commands and limits            PASS
T26  Frame content is deterministic                 PASS
T27  Timer ISR touches only the tick                PASS
T28  Scheduler accuracy (no drift)                  PASS
T29  UART fuzzing (100000 bytes)                    PASS
T30  Worst-case telemetry length                    PASS
T31  Randomised soak, fault injection               PASS
T32  Alert cleared when its sensor fails            PASS
Tests passed: 32/32   Checks executed: 163   Checks failed: 0
```

Appendix C: 30-Day Soak Report

```
Soak test: 720 simulated hours, seed 20261014, 12847 random events, 223 injected resets
  telemetry messages: 44879, watchdog resets: 0
  I1 outputs OFF when idle                violations: 0
  I2 alert => LED on                      violations: 0
  I3 alert => state ALERT                 violations: 0
  I4 no temp/hum alert during DHT fault   violations: 0
  I5 no AIR alert during MQ fault         violations: 0
  I6 ring buffer indices in range         violations: 0
  I7 OLED lines terminated                violations: 0
  I8 NORMAL => no alerts                  violations: 0
  I9 threshold ordering                   violations: 0
  I10 payload within buffer               violations: 0
  I11 sequence numbers increase           violations: 0
```

Appendix D: Glossary

Term	Meaning
ESP32	Low-cost microcontroller with Wi-Fi, used as the system's brain
Firmware	The program that runs on the microcontroller
Interrupt (ISR)	A routine the processor runs immediately when an event such as a timer tick occurs; it must be very short
HAL	Hardware abstraction layer: the small set of functions that touch the hardware, so the rest of the code does not depend on it
Hysteresis	Using different switch-on and switch-off points so an alert does not flicker near the limit
Watchdog	A timer that resets the device if the program stops responding
Soak test	A long test with random events that looks for rare failures
Fuzzing	Feeding random input to find crashes
Mutation testing	Injecting known bugs to check whether the tests notice them
Valgrind	A tool that finds memory errors while a program runs
Code coverage	The share of code lines that tests actually execute
Simulation	Running the real firmware logic on a PC with a software model of the hardware